

# High Performance Computing

## Lab 3: Parallel Program Design and Monte Carlo Simulation

The purpose of this lab is to practice the basic principles of parallel program design using a simple computational problem. You will implement a Monte Carlo approximation of  $\pi$ , divide the computation into independent tasks, combine partial results using a reduction operation, and analyze the problem in terms of partitioning, communication, agglomeration, and mapping.

No parallel programming library is required for this lab.

### Part 1: Monte Carlo Approximation of $\pi$

Consider a square with side length 2 and a circle of radius 1 inside the square.

Random points  $(x, y)$  are generated inside the square:

$$-1 \leq x \leq 1, \quad -1 \leq y \leq 1.$$

A point is inside the circle if

$$x^2 + y^2 \leq 1.$$

Let

$$N$$

be the total number of generated points and let

$$M$$

be the number of points that fall inside the circle.

The value of  $\pi$  can be approximated as

$$\pi \approx 4 \frac{M}{N}.$$

Write a C++ program that generates random points and calculates an approximation of  $\pi$ .

You may start with the following code:

```
1  #include <iostream>
2  #include <random>
3
4  int main() {
5      const long long N = 1000000;
6
7      std::mt19937 generator(42);
8      std::uniform_real_distribution<double> distribution(-1.0, 1.0);
9
10     long long inside = 0;
11
12     for (long long i = 0; i < N; ++i) {
13         double x = distribution(generator)
```

```

14     double y = distribution(generator);
        if (x * x + y * y <= 1.0) {
            inside++;
        }
    }

    double pi = 4.0 * inside / N;

    std::cout << "Total points: " << N << std::endl;
    std::cout << "Points inside circle: " << inside <<
std::endl;
    std::cout << "Approximated pi: " << pi << std::endl;

    return 0;
}
15
16
17
18
19
20
21
22
23
24
25 }

```

Your program should print:

- the total number of generated points;
- the number of points inside the circle;
- the approximated value of  $\pi$ .

The screenshot shows a C++ IDE with the following components:

- Source Code (Left Panel):**

```

1 #include <iostream>
2 #include <random>
3
4 int main() {
5     const long long N = 1000000;
6
7     std::mt19937 generator(42);
8     std::uniform_real_distribution<double> distribution(-1.0, 1.0);
9
10    long long inside = 0;
11
12    for (long long i = 0; i < N; ++i) {
13        double x = distribution(generator);
14        double y = distribution(generator);
15
16        if (x * x + y * y <= 1.0) {
17            inside++;
18        }
19    }
20
21    double pi = 4.0 * inside / N;
22
23    std::cout << "Total points: " << N << std::endl;
24    std::cout << "Points inside circle: " << inside << std::endl;
25    std::cout << "Approximated pi: " << pi << std::endl;
26
27    return 0;
28 }

```
- Execution Output (Right Panel):**

```

x86-64 gcc 16.2
Program returned: 0
Program stdout:
Total points: 1000000
Points inside circle: 785607
Approximated pi: 3.14243

```

## Part 2: Effect of Problem Size

Run the program using the following values:

$$N = 10^3, \quad N = 10^4, \quad N = 10^5, \quad N = 10^6.$$

Record the approximated value of  $\pi$ .

| $N$    | Approximation of $\pi$ |
|--------|------------------------|
| $10^3$ | 3.152                  |
| $10^4$ | 3.1708                 |
| $10^5$ | 3.14668                |
| $10^6$ | 3.14243                |

Answer the following questions.

1. Does the approximation generally improve when  $N$  increases?

Yes. The more points we generate, the closer the result gets to the real value of  $\pi$  (3.14159...). This is how Monte Carlo works, more samples = better accuracy

2. Does the program have to store all generated points?

No. We only need to keep a counter. Each point is checked immediately and then discarded, we never need to look at it again

3. Can one generated point be processed without knowing the result for another point?

Yes. Every point is completely independent. Whether point #500 is inside the circle has nothing to do with point #1 or point #999

### Part 3: Partitioning

Suppose that four processors are available:

$$P_0, \quad P_1, \quad P_2, \quad P_3.$$

Assume

$$N = 1,000,000.$$

Divide the generated points equally among the four processors.  
Complete the following table.

| Processor | Number of Points | Local Result |
|-----------|------------------|--------------|
| $P_0$     | 250 000          | $M_0$        |
| $P_1$     | 250 000          | $M_1$        |
| $P_2$     | 250 000          | $M_2$        |
| $P_3$     | 250 000          | $M_3$        |

Answer:

1. How many points should each processor generate?

250,000 (= 1,000,000 / 4)

2. Can the processors perform their work independently?

Yes. Each processor only needs to generate its own points and count how many fall inside the circle. No communication is needed during computation

3. Is this problem an example of domain decomposition or functional decomposition?

Domain decomposition. We are splitting the data (the points) across processors, not splitting different functions/tasks

4. Does one processor need the coordinates generated by another processor?

No. Each processor generates its own random points from scratch and never shares them

## Part 4: Simulating Multiple Processors

Modify your program so that it simulates four processors.

The program should calculate a separate local count for each processor.

For example:

```

1  const int P = 4;
2
3  long long local_inside[P] = {0};
4
5  for (int p = 0; p < P; ++p) {
6
7      for (long long i = 0; i < N / P; ++i) {
8
9          double x = distribution(generator);
10         double y = distribution(generator);
11
12         if (x * x + y * y <= 1.0) {
13             local_inside[p]++;
14         }
15     }
16 }
```

Print the local result of every simulated processor:

Processor 0: 196427

Processor 1: 196282

Processor 2: 196271

Processor 3: 196627

Note that this program is still executed serially. The purpose is to represent how the problem could be divided before introducing an actual parallel programming library.

## Part 5: Reduction

After all processors finish their local computations, the total number of points inside the circle must be calculated as

$$M = M_0 + M_1 + M_2 + M_3.$$

Add this reduction operation to your program.

For example:

```

1 long long total_inside = 0;
2
3 for (int p = 0; p < P; ++p) {
4     total_inside += local_inside[p];
5 }

```

Then calculate

$$\pi \approx 4 \frac{M}{N}.$$

Total inside: 785607

Final pi: 3.14243

Answer the following questions.

1. Is calculating each  $M_p$  an independent computation?

Yes. Each processor computes its own local count without needing any result from others

2. Is calculating the final  $M$  an independent computation?

No. We must wait for all four processors to finish before we can sum them up

3. What information must processors communicate at the end?

Each processor sends only one number — its local count  $M_p$ . That's it. No coordinates, no intermediate results

4. Is the final summation an example of local communication or global communication?

Global communication. All processors are involved — every local result must reach the one processor doing the sum

5. What is this operation called in parallel computing?

A reduction operation (specifically, sum reduction). In MPI it would be MPI\_Reduce with MPI\_SUM

Global communication. All processors are involved — every local result must reach the one processor doing the sum

```

7  std::mt19937 generator(42);
8  std::uniform_real_distribution<double> distribution(-1.0, 1.0);
9
10 long long local_inside[P] = {0};
11
12 for (int p = 0; p < P; ++p) {
13     for (long long i = 0; i < N / P; ++i) {
14         double x = distribution(generator);
15         double y = distribution(generator);
16
17         if (x * x + y * y <= 1.0) {
18             local_inside[p]++;
19         }
20     }
21 }
22
23 for (int p = 0; p < P; ++p) {
24     std::cout << "Processor " << p << ": " << local_inside[p] << std::endl;
25 }
26
27 // Part 5: Reduction
28 long long total_inside = 0;
29 for (int p = 0; p < P; ++p) {
30     total_inside += local_inside[p];
31 }
32
33 std::cout << "Total inside: " << total_inside << std::endl;
34 std::cout << "Final pi: " << 4.0 * total_inside / N << std::endl;
35
36 return 0;
37 }

```

Program returned: 0

Program stdout:

```

Processor 0: 196427
Processor 1: 196282
Processor 2: 196271
Processor 3: 196627
Total inside: 785807
Final pi: 3.14243

```

## Part 6: Agglomeration

Suppose we initially create one task for every generated point.

For

$$N = 1,000,000,$$

this would create one million very small tasks.

Instead, we can combine many points into larger tasks.

For example, one task could process

$$10,000$$

points.

Answer:

1. How many tasks would be created if each task processes 10,000 points?

100 tasks (= 1,000,000 / 10,000)

2. Is one task per point an example of fine-grained or coarse-grained parallelism?

Fine-grained. Each task does almost no work, so the overhead of managing it is larger than the actual computation

3. Why can combining many small tasks into larger tasks reduce overhead?

Every task has a cost: scheduling, memory allocation, synchronization. When tasks are tiny, this overhead dominates. Bigger tasks mean fewer coordination events and much better efficiency

4. What disadvantage could occur if the tasks become too large?

Load imbalance. If we have fewer tasks than processors, some processors sit idle. Also, if one task takes much longer than others, everyone waits for it

## Part 7: Mapping

Assume that eight equal tasks are created:

$T_0, T_1, T_2, T_3, T_4, T_5, T_6, T_7.$

There are four processors:

$P_0, P_1, P_2, P_3.$

Assign the tasks so that each processor receives approximately the same amount of work.  
Complete the table.

| Processor | Assigned Tasks |
|-----------|----------------|
| $P_0$     |                |
| $P_1$     |                |
| $P_2$     |                |
| $P_3$     |                |

  

| Processor | Assigned Tasks |
|-----------|----------------|
| P0        | T0, T1         |
| P1        | T2, T3         |
| P2        | T4, T5         |
| P3        | T6, T7         |

Answer:

1. How many tasks does each processor receive?

2 tasks (= 8 / 4)

2. Is the workload balanced?

Yes. All tasks are equal size and each processor gets exactly 2 tasks

3. What happens to the total execution time if one processor receives much more work than the others?

The total execution time is limited by the slowest processor. The others finish early and just wait — wasted time, wasted resources. This is called load imbalance

## Part 8: Parallel Program Design

For the Monte Carlo problem, briefly describe each of the following steps.

### 1. Partitioning:

What should be divided into independent tasks?

Divide the N point-generation + circle-check operations into independent tasks. Each point is a natural unit of work with zero dependencies on other points

### 2. Communication:

What information needs to be communicated between tasks?

Only the local counts  $M_p$  need to be communicated, one integer per processor, sent at the very end. The (x, y) coordinates never leave the processor that created them

### 3. Agglomeration:

Why could many individual point-generation tasks be combined into larger tasks?

One task per point = 1 000 000 tasks with near-zero computation each. We combine them into P larger tasks (one per processor), each processing N/P points. This eliminates task management overhead entirely

### 4. Mapping:

How should these tasks be assigned to available processors?

Assign one task to each processor, equal block size guarantees perfect load balance since every point takes the same time to compute

## Part 9: Short Analysis

Consider the main computation:

```

1 for (long long i = 0; i < N; ++i) {
    double x = distribution(generator);
    double y = distribution(generator);
  }

```

2  
3  
4  
5  
6  
7  
8  
9

Answer the following questions.

1. What is the time complexity of this loop in Big-O notation?

$O(N)$ . The loop runs exactly  $N$  times, and each iteration does a fixed number of operations

2. If  $N$  is doubled, approximately how much additional work must be performed?

Exactly twice as much.  $O(N)$  is linear. Double the input, double the work

3. Which part of the algorithm can be executed independently by different processors?

The main loop, generating points and checking if they're inside the circle. Each iteration is fully independent

4. Which operation requires the results from all processors?

The final reduction:  $M = M_0 + M_1 + M_2 + M_3$ . This step cannot start until every processor has finished its local computation

In this lab I implemented a Monte Carlo simulation to approximate  $\pi$  and analyzed it using the PCAM parallel design methodology. I learned that the problem is embarrassingly parallel. Each point can be processed completely independently, which means it scales almost perfectly with the number of processors. The accuracy improves as  $O(1/\text{root of } N)$ , meaning we need 100 x more points for 10 x better precision. The most important insight was the role of agglomeration: without it, 1 000 000 tiny tasks would create more overhead than actual work. Finally, the reduction step showed how even a fully parallel problem requires a small amount of global communication at the end to combine results

## Submission

Submit:

1. the C++ source code;
2. the results for different values of  $N$ ;
3. the output showing the local results of four simulated processors;
4. the final approximation of  $\pi$  after reduction;
5. completed tables and answers to all questions.